

PROJECT DOCUMENTATION

Title: AI Software Development Team using Multi-Agent Architecture

1. Introduction

This project presents an AI-driven system that simulates a complete software development team using multiple specialized agents. Each agent represents a specific role in the software development lifecycle, such as Product Manager, Architect, Developer, Reviewer, QA Engineer, and DevOps Engineer.

The system leverages **Large Language Models (LLMs) via AWS Bedrock** and orchestrates interactions between agents to automatically generate software solutions from a given idea. The architecture demonstrates how multi-agent systems can collaborate to perform complex tasks such as requirement analysis, system design, coding, testing, and deployment planning.

2. Objectives

- To design a multi-agent AI system for automated software development
- To simulate real-world software engineering roles using LLM-based agents
- To implement an orchestrator that coordinates agent interactions
- To integrate memory using vector databases for contextual continuity
- To generate end-to-end project outputs from a single idea
- To provide a user interface for interacting with the system

3. Technologies and Libraries Used

3.1 Technologies

- Python: Core programming language
- FastAPI: Backend API service
- Streamlit: Frontend interface
- AWS Bedrock: LLM inference
- FAISS: Vector database for memory

3.2 Libraries

- boto3: AWS SDK for Bedrock access
- langchain_community.vectorstores: FAISS integration
- langchain_community.embeddings: HuggingFace embeddings
- pydantic: Data validation
- requests: API communication

4. System Architecture

4.1 Workflow

User Idea → Orchestrator → Product Manager → Architect → Backend Developer → Code Reviewer → Frontend Developer → QA Engineer → DevOps Engineer → Final Output

4.2 Component Description

1. Base Agent

- Core abstraction for all agents
- Handles prompt construction and LLM invocation
- Ensures consistent structure and rules across agents

2. Specialized Agents

- **Product Manager Agent**
Generates requirements, user stories, and feature definitions
- **Architect Agent**
Designs system architecture, APIs, and database schema
- **Backend Developer Agent**
Generates backend code
- **Frontend Developer Agent**
Builds UI using modern frameworks
- **Code Reviewer Agent**
Reviews and improves generated code
- **QA Agent**
Generates test cases and edge scenarios
- **DevOps Agent**
Provides deployment strategies and CI/CD pipelines

3. Orchestrator (SoftwareTeamOrchestrator)

- Central controller of the system
- Executes agents sequentially
- Passes outputs between agents
- Implements iterative improvement via review loops
- Stores intermediate outputs in memory

4. Memory System (Vector Store)

- Uses FAISS for semantic storage
- Stores outputs such as requirements and architecture
- Enables retrieval of relevant past context
- Improves coherence across agent outputs

5. Backend (FastAPI)

- Exposes /build endpoint
- Accepts project idea
- Returns complete generated solution

6. Frontend (Streamlit)

- Accepts user input (project idea)
- Displays outputs from each agent stage
- Provides structured visualization of generated results

5. Features

- Multi-agent collaboration system
- End-to-end software generation from a single idea
- Role-based agent specialization
- Iterative code refinement using reviewer loop
- Context-aware memory using vector store
- Backend API with FastAPI
- Interactive UI with Streamlit

6. Workflow Execution Details

1. User submits a project idea
2. Product Manager generates requirements
3. Architect designs system structure
4. Backend Developer generates backend code
5. Code Reviewer reviews and suggests improvements

6. Backend code is refined iteratively
7. Frontend Developer generates UI code
8. QA Agent produces test cases
9. DevOps Agent provides deployment strategy
10. Final output is returned

7. Code Implementation

```
from agents.product_manager import ProductManagerAgent
from agents.architect import ArchitectAgent
from agents.backend_dev import BackendDevAgent
from agents.frontend_dev import FrontendDevAgent
from agents.code_reviewer import CodeReviewerAgent
from agents.qa_agent import QAAgent
from agents.devops_agent import DevOpsAgent
```

```
from tools.vector_store import VectorStore #  ADD THIS
```

```
def safe_run(agent, input_text):
    try:
        result = agent.run(input_text)
        if not result or len(result.strip()) < 20:
            return "⚠️ Agent failed to generate valid output"
        return result
    except Exception as e:
        return f"Error: {str(e)}"
```

```
class SoftwareTeamOrchestrator:
```

```
    def __init__(self):
        self.pm = ProductManagerAgent()
        self.architect = ArchitectAgent()
```

```
self.backend = BackendDevAgent()
self.frontend = FrontendDevAgent()
self.reviewer = CodeReviewerAgent()
self.qa = QAAgent()
self.devops = DevOpsAgent()
```

```
self.memory = VectorStore() # ✅ INIT MEMORY
```

```
def run_pipeline(self, idea):
```

```
    context = {
        "idea": idea
    }
```

```
    # 🚩 Step 1: Requirements
```

```
    context["requirements"] = safe_run(self.pm, context["idea"])
```

```
    self.memory.add_text(context["requirements"]) # ✅ STORE
```

```
    # 🔍 Retrieve memory (optional but powerful)
```

```
    past_context = self.memory.search("software requirements")
```

```
    # 🏗️ Step 2: Architecture
```

```
    context["architecture"] = safe_run(
        self.architect,
        f"""
```

Use these requirements:

```
{context["requirements"]}
```

Relevant past knowledge:

```
{past_context}
```

Do NOT change domain.

```
"""
```

```
)
```

```
self.memory.add_text(context["architecture"]) # ✅ STORE
```

```
# 🔄 Step 3: Backend
```

```
backend_input = f"""
```

Use this architecture strictly:

```
{context["architecture"]}
```

Relevant past knowledge:

```
{past_context}
```

RULES:

- Use FastAPI only

- Keep consistency

```
"""
```

```
context["backend_code"] = safe_run(self.backend, backend_input)
```

```
self.memory.add_text(context["backend_code"]) # ✅ STORE
```

```
# 🔄 Step 4: Reviewer Loop
```

```
for _ in range(2):
```

```
    review = safe_run(self.reviewer, context["backend_code"])
```

```
    if "no issues" in review.lower():
```

```
        break
```

```
fix_prompt = f"""
```

Fix the code based on review.

CODE:

```
{context["backend_code"]}
```

REVIEW:

```
{review}
```

```
"""
```

```
context["backend_code"] = safe_run(self.backend, fix_prompt)
```

```
context["review"] = review
```

```
# 🎨 Step 5: Frontend
```

```
context["frontend_code"] = safe_run(
    self.frontend,
    f"""
```

Build UI based on backend:

```
{context["backend_code"]}
```

```
"""
```

```
)
```

```
# 🧪 Step 6: QA
```

```
context["tests"] = safe_run(self.qa, context["backend_code"])
```

```
# 🚀 Step 7: DevOps
```

```
context["deployment"] = safe_run(
    self.devops,
    f"""
```

Deploy system:

ARCHITECTURE:

```
{context["architecture"]}
```

BACKEND:

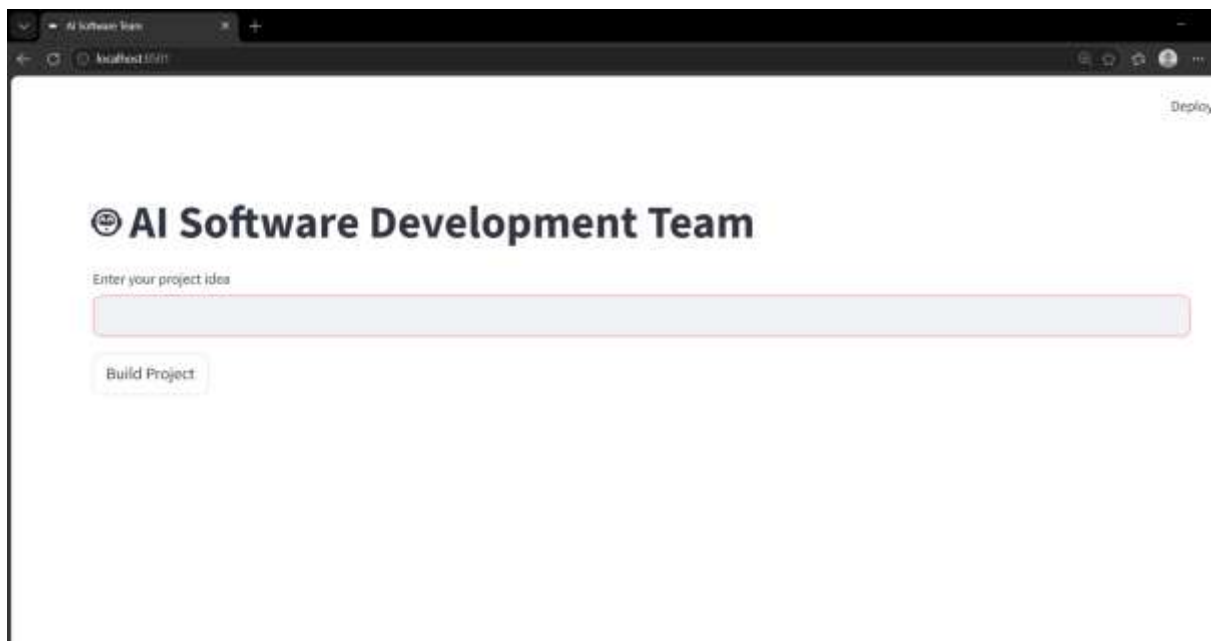
```
{context["backend_code"]}
```

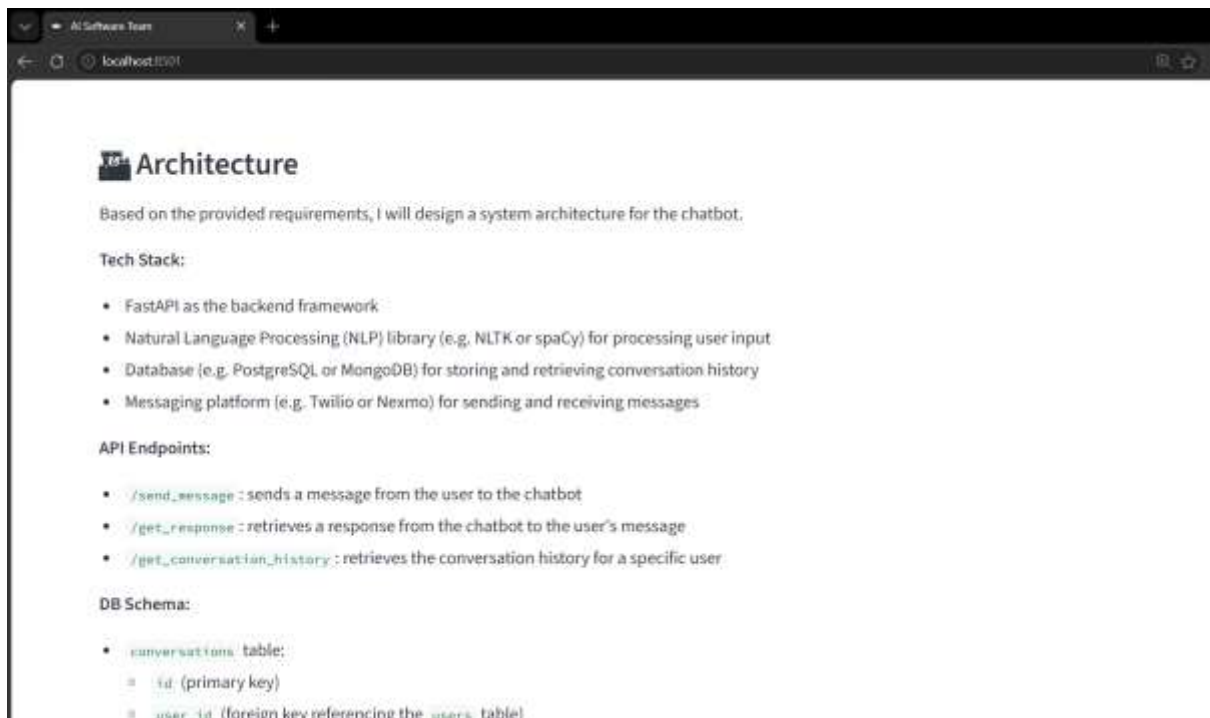
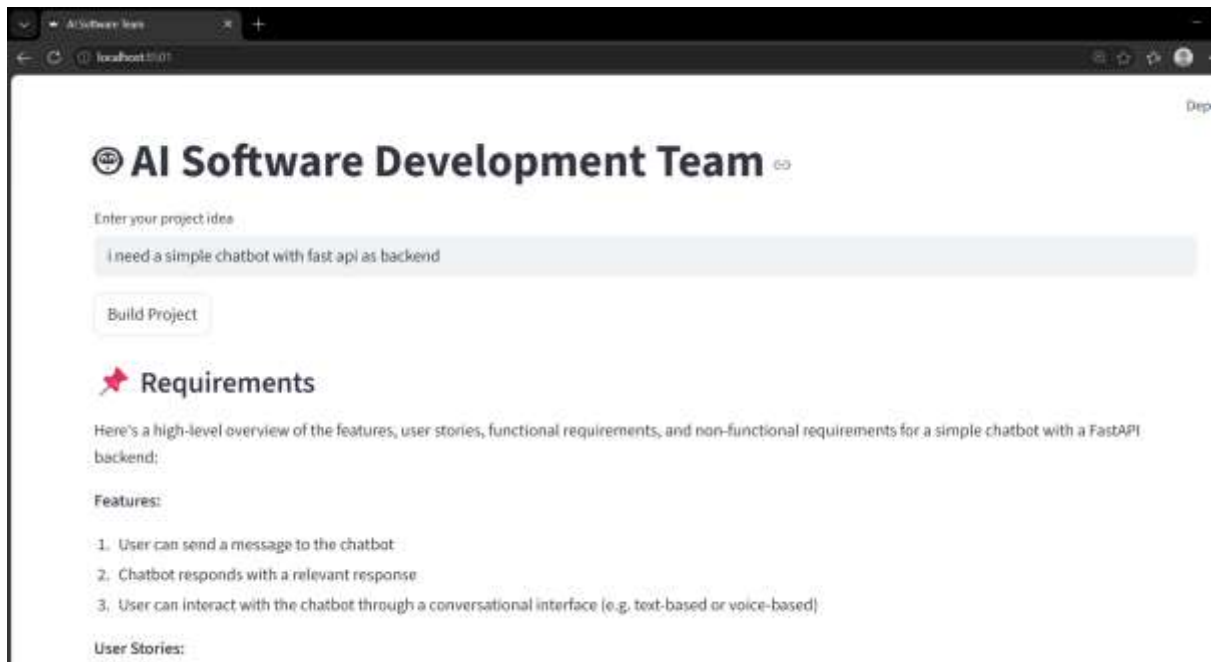
```
"""
```

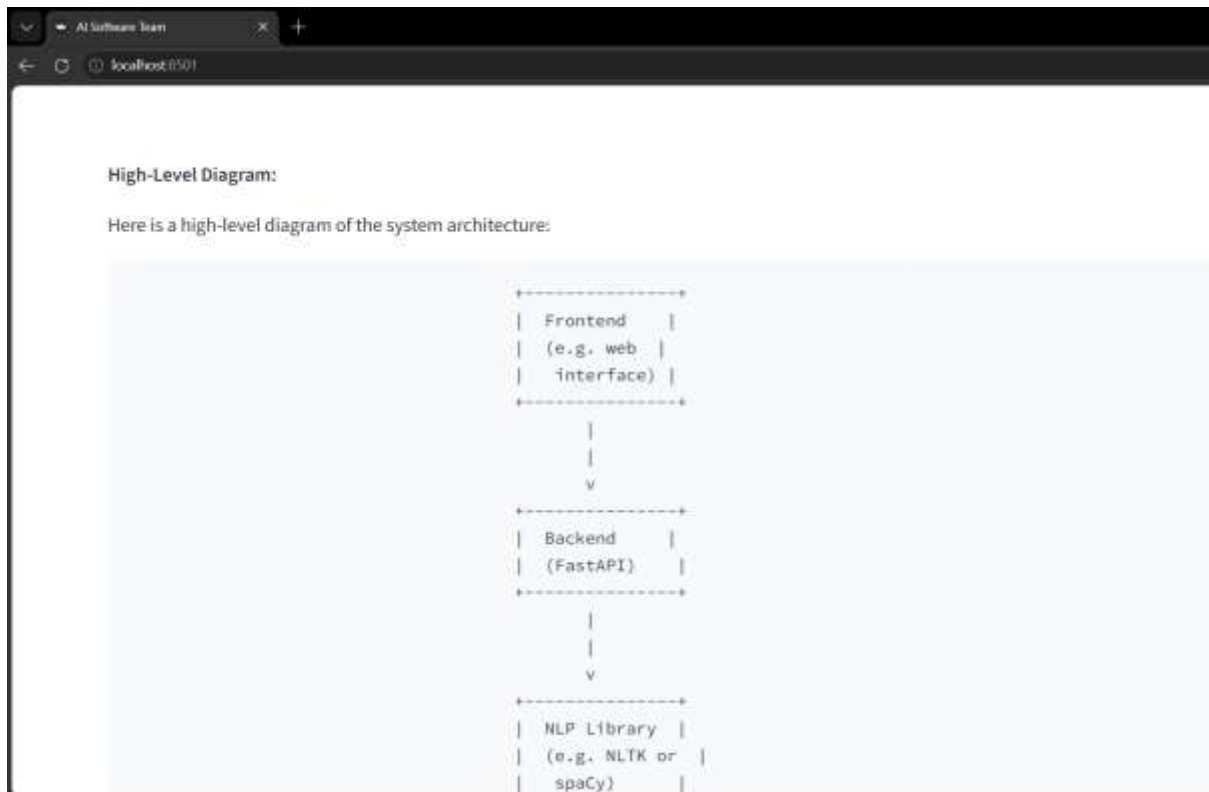
```
)
```

```
return context
```

8. Output







Backend Code

Here is the refactored code based on the review:

```
"""main.py"""
python
from fastapi import FastAPI, HTTPException
from fastapi.responses import JSONResponse
from fastapi.requests import Request
from pydantic import BaseModel
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer
from typing import List
from database import get_conversation_history, get_conversation_history_by_user_id
from nlp import process_user_input

app = FastAPI()

class Message(BaseModel):
    message: str

class Response(BaseModel):
```

AI System View

localhost:3001

Frontend Code

Here is a modern UI design using React:

```
==index.html==
<!--html
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Chatbot UI</title>
  <link rel="stylesheet" href="styles.css" />
</head>
<body>
  <div id="root"></div>
  <script src="index.js"></script>
</body>
</html>
...
==styles.css==
/*css
```

localhost:3001

Review

The refactored code looks good! Here are some minor suggestions for further improvement:

main.py

1. In the `send_message` endpoint, you can add a check to ensure that the `user_input` is not empty before processing it. This will prevent the API from returning an empty response.

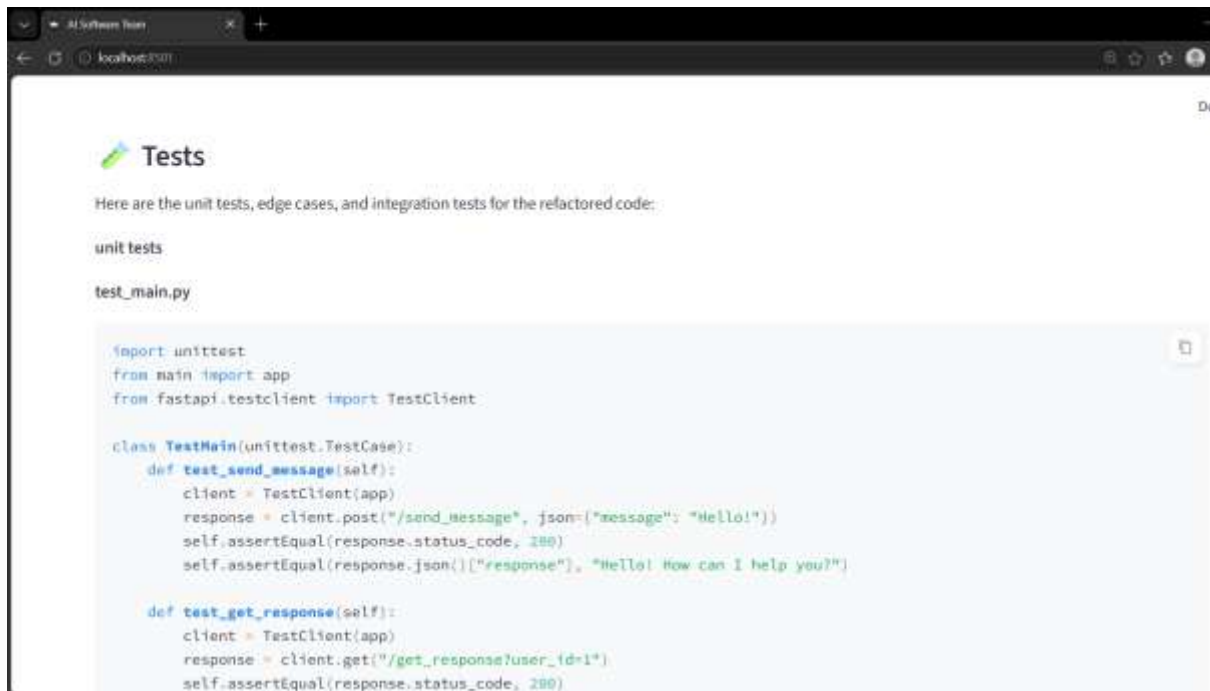
```
if not user_input:
    raise HTTPException(status_code=400, detail="Invalid user input")
```

2. In the `get_response` endpoint, you can add a check to ensure that the `user_id` is not None before querying the database. This will prevent the API from returning an error when no user ID is provided.

```
if user_id is None:
    raise HTTPException(status_code=400, detail="User ID is required")
```

database.py

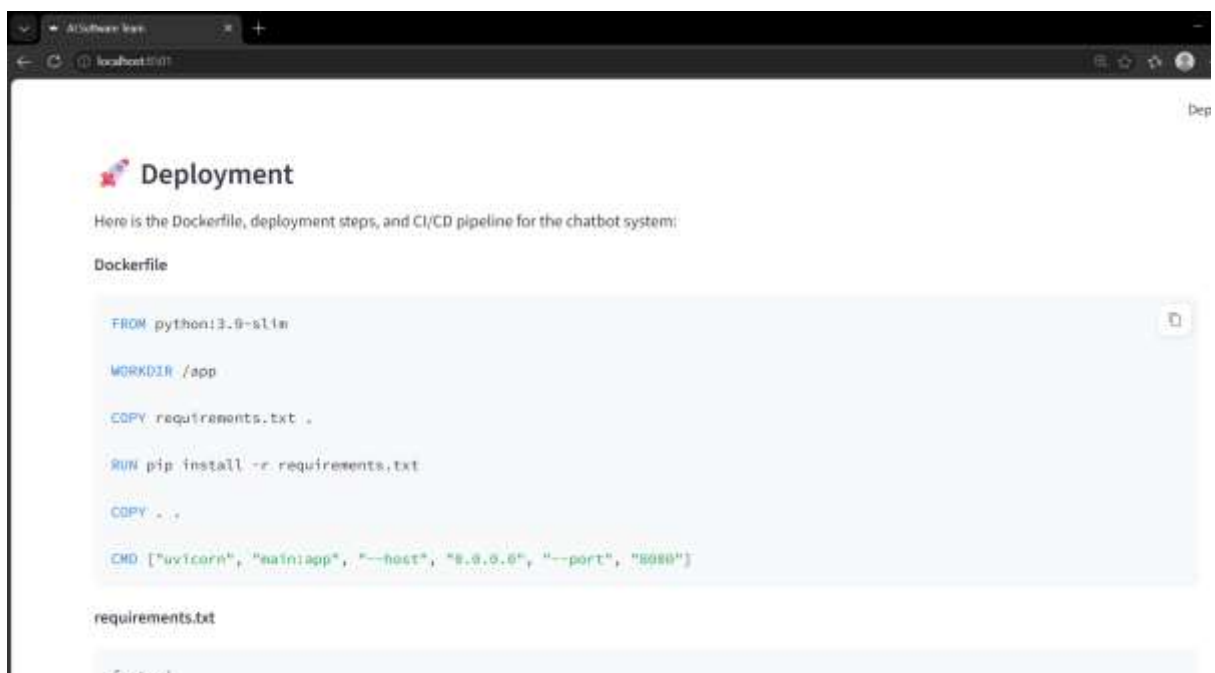
1. You can add a try-except block to the `get_conversation_history` and `get_conversation_history_by_user_id` functions to handle database connection



```
import unittest
from main import app
from fastapi.testclient import TestClient

class TestMain(unittest.TestCase):
    def test_send_message(self):
        client = TestClient(app)
        response = client.post("/send_message", json={"message": "Hello!"})
        self.assertEqual(response.status_code, 200)
        self.assertEqual(response.json()["response"], "Hello! How can I help you?")

    def test_get_response(self):
        client = TestClient(app)
        response = client.get("/get_response?user_id=1")
        self.assertEqual(response.status_code, 200)
```



```
FROM python:3.9-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install -r requirements.txt

COPY . .

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8080"]
```

requirements.txt

```
fastapi
```

9. Conclusion

This project demonstrates the potential of multi-agent systems in automating software development processes. By combining role-based agents with LLM capabilities and memory systems, the platform can generate structured and coherent outputs across multiple stages of development. The modular design allows scalability and further enhancement toward intelligent autonomous software engineering systems.